

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



MÔN HỌC: THIẾT KẾ LUẬN LÝ VỚI HDL
MÃ MÔN HỌC: CO1025
HỌC KỲ: 252

BÁO CÁO ĐỒ ÁN MÔN HỌC

THIẾT KẾ RISC CPU ĐƠN GIẢN

Giảng viên hướng dẫn: HUỖNH PHÚC NGHỊ

Lớp: L01

Nhóm thực hiện: 9

DANH SÁCH THÀNH VIÊN

STT	Họ và tên	MSSV
1	PHẠM ĐỖ HỒNG PHÚC	2510359
2	TRẦN TIẾN THÀNH	2413182
3	VÔ LÊ PHÚC THỊNH	2510382

TP. HỒ CHÍ MINH, THÁNG 4 NĂM 2026

Mục lục

1	GIỚI THIỆU	1
1.1	Tổng quan đề tài	1
1.2	Mục tiêu của nhóm	1
1.3	Lý do chọn đề tài	2
1.4	Công cụ sử dụng	2
1.5	Thiết bị sử dụng	2
1.6	Các chức năng của sản phẩm	2
2	LÝ THUYẾT SƠ LƯỢC	4
2.1	Kiến trúc RISC	4
2.2	Tổ chức CPU đơn giản	5
2.3	Tập lệnh và mã hóa lệnh	5
2.4	Ứng dụng thực tế của kiến trúc RISC	6
3	THIẾT KẾ	8
3.1	Các khối chức năng	8
3.1.1	Program Counter (PC)	8
3.1.2	Address Mux	9
3.1.3	ALU (Arithmetic Logic Unit)	10
3.1.4	Register (Accumulator / Instruction Register)	13
3.1.5	Memory	14
4	HIỆN THỰC	16
4.1	Hiện thực ALU	16
4.2	Hiện thực Register	17
4.2.1	Hiện thực Accumulator Register	17
5	KIỂM THỬ VÀ ĐÁNH GIÁ	19
5.1	Testbench từng module	19
5.1.1	Testbench và Kết quả mô phỏng ALU	19
5.1.2	Testbench và Kết quả mô phỏng Accumulator Register	23
6	KẾT LUẬN	27
6.1	Kết quả đạt được	27
6.2	Khó khăn và Thách thức	27
6.3	Hướng phát triển và Mở rộng	27
6.4	Bảng phân chia công việc	28

1 GIỚI THIỆU

RISC (Reduced Instruction Set Computer) là một phương pháp thiết kế bộ xử lý hiện đại, tập trung vào việc giảm thiểu tập lệnh nhằm tăng tốc độ thực thi. Trong đề tài này, nhóm thực hiện thiết kế một RISC CPU đơn giản với 3-bit opcode và 5-bit toán hạng, tương ứng với 8 loại câu lệnh và 32 không gian địa chỉ.

1.1 Tổng quan đề tài

Trong bối cảnh ngành công nghiệp vi mạch và hệ thống nhúng đang phát triển mạnh mẽ, việc nắm vững nguyên lý thiết kế bộ xử lý từ cấp độ phần cứng là một nền tảng quan trọng đối với mỗi kỹ sư điện tử và kỹ thuật máy tính. Đề tài "Thiết kế RISC CPU đơn giản" được thực hiện nhằm mục tiêu xây dựng một bộ xử lý thu gọn, có thể mô phỏng và kiểm chứng được, đủ để minh họa toàn bộ vòng đời thực thi lệnh — từ giai đoạn nạp lệnh (fetch), giải mã (decode), thực thi (execute) đến ghi kết quả (write-back).

CPU được thiết kế trong đề tài sử dụng định dạng lệnh 8-bit, trong đó:

- **3-bit opcode:** cho phép mã hóa tối đa 8 lệnh khác nhau.
- **5-bit toán hạng:** cung cấp 32 địa chỉ khả dụng cho bộ nhớ và thanh ghi.

Kiến trúc tối giản này không chỉ phù hợp với mục tiêu học thuật mà còn phản ánh đúng triết lý cốt lõi của RISC: ít lệnh hơn, thực thi nhanh hơn, phần cứng đơn giản hơn.

1.2 Mục tiêu của nhóm

Nhóm đặt ra các mục tiêu cụ thể sau cho đề tài:

1. **Hiểu sâu kiến trúc RISC** — Nắm vững nguyên lý hoạt động của bộ xử lý kiểu RISC, phân biệt với kiến trúc CISC, và hiểu lý do RISC được ưu tiên trong các hệ thống nhúng và thiết bị di động hiện đại.
2. **Thiết kế tập lệnh (ISA)** — Xây dựng một tập lệnh gồm 8 lệnh cơ bản, bao gồm các lệnh số học, logic, truy xuất bộ nhớ và điều khiển luồng chương trình.
3. **Triển khai phần cứng bằng HDL** — Mô tả toàn bộ datapath và bộ điều khiển (control unit) bằng ngôn ngữ mô tả phần cứng (Verilog/VHDL), đảm bảo có thể tổng hợp và mô phỏng được.
4. **Kiểm thử và xác minh** — Viết các testbench để kiểm tra tính đúng đắn của từng khối chức năng (ALU, thanh ghi, bộ nhớ, bộ giải mã lệnh) cũng như toàn bộ hệ thống.
5. **Trình bày kết quả rõ ràng** — Tài liệu hóa quá trình thiết kế, sơ đồ khối, dạng sóng mô phỏng và phân tích kết quả một cách có hệ thống.

1.3 Lý do chọn đề tài

Nhóm lựa chọn đề tài thiết kế RISC CPU xuất phát từ ba lý do chính:

Thứ nhất, giá trị học thuật cao. Bộ xử lý là trái tim của mọi hệ thống máy tính. Việc tự tay thiết kế một CPU — dù ở quy mô nhỏ — giúp chúng tôi hiểu thực sự cách phần cứng và phần mềm tương tác với nhau, thay vì chỉ tiếp cận lý thuyết qua sách vở.

Thứ hai, tính thực tiễn và ứng dụng. Kiến trúc RISC là nền tảng của các bộ xử lý phổ biến nhất hiện nay như ARM (dùng trong smartphone), RISC-V (đang được áp dụng rộng rãi trong nghiên cứu và công nghiệp). Nắm vững RISC là bước khởi đầu thiết thực để tiếp cận các kiến trúc thực tế này.

Thứ ba, phạm vi phù hợp với thời gian và nguồn lực. Với định dạng lệnh 8-bit và tập lệnh giới hạn ở 8 lệnh, đề tài có độ phức tạp vừa đủ — đủ thách thức để học hỏi sâu, nhưng không quá lớn để mất kiểm soát trong khuôn khổ học phần. Điều này cho phép nhóm tập trung vào chất lượng thiết kế thay vì chạy theo số lượng tính năng.

Với những định hướng trên, nhóm kỳ vọng đề tài không chỉ là một bài tập kỹ thuật hoàn chỉnh, mà còn là nền tảng vững chắc để mỗi thành viên tự tin tiếp cận các hệ thống vi xử lý phức tạp hơn trong tương lai.

1.4 Công cụ sử dụng

- **Xilinx Vivado Design Suite:** Môi trường thiết kế tích hợp (IDE) chuyên dụng cho FPGA của Xilinx, được sử dụng để viết mã Verilog/VHDL, tổng hợp (synthesis), mô phỏng (simulation) và kiểm tra dạng sóng (waveform) cho toàn bộ thiết kế RISC CPU.
- **Draw.io (diagrams.net):** Công cụ vẽ sơ đồ trực tuyến miễn phí, được sử dụng để thiết kế và trình bày các sơ đồ khối kiến trúc CPU, datapath, sơ đồ trạng thái (FSM) và các lưu đồ giải thuật trong báo cáo.

1.5 Thiết bị sử dụng

- **Máy tính cá nhân (PC/Laptop):** Được sử dụng để chạy phần mềm Xilinx Vivado, viết mã HDL, thực hiện mô phỏng và kiểm tra kết quả thiết kế thông qua waveform simulation.
- **Bo mạch FPGA Digilent Arty Z7:** Bo mạch phát triển tích hợp chip Xilinx Zynq-7000 SoC (XC7Z020), kết hợp giữa bộ xử lý ARM Cortex-A9 và phần logic khả trình FPGA. Bo mạch được sử dụng để nạp (program) và kiểm thử thiết kế RISC CPU trực tiếp trên phần cứng thực tế, xác minh tính đúng đắn của hệ thống ngoài môi trường mô phỏng.

1.6 Các chức năng của sản phẩm

Hệ thống RISC CPU được thiết kế bao gồm các chức năng cốt lõi sau:

- **Nạp lệnh (Instruction Fetch):** Lấy lệnh tiếp theo từ bộ nhớ.
- **Giải mã lệnh (Instruction Decode):** Phân tích mã lệnh để xác định thao tác cần thực hiện.
- **Lấy dữ liệu toán hạng (Operand Fetch):** Truy xuất dữ liệu từ bộ nhớ (nếu cần).
- **Thực thi câu lệnh (Execute):** Xử lý các phép toán thông qua khối ALU.
- **Lưu trữ kết quả (Store):** Ghi dữ liệu về bộ nhớ hoặc Accumulator.

Tập lệnh hỗ trợ: Hệ thống được thiết kế để tự động dừng chương trình khi gặp lệnh HALT và hỗ trợ 8 lệnh cơ bản như bảng sau:

STT	Lệnh	Chức năng
1	HLT	Dừng chương trình
2	SKZ	Bỏ qua lệnh tiếp theo nếu Accumulator = 0
3	ADD	Cộng dữ liệu
4	AND	Phép toán AND logic
5	XOR	Phép toán XOR logic
6	LDA	Nạp dữ liệu vào Accumulator
7	STO	Lưu dữ liệu từ Accumulator ra bộ nhớ
8	JMP	Nhảy đến địa chỉ chỉ định

Bảng 1: Tập lệnh của RISC CPU

2 LÝ THUYẾT SƠ LƯỢC

2.1 Kiến trúc RISC

RISC (Reduced Instruction Set Computer – Máy tính với tập lệnh rút gọn) là triết lý thiết kế bộ vi xử lý hướng tới việc đơn giản hóa tập lệnh, giúp mỗi lệnh có thể thực thi trong một chu kỳ đồng hồ (single clock cycle). Kiến trúc RISC xuất hiện từ những năm 1980 với các dự án tiên phong tại Đại học Berkeley và Stanford, trở thành nền tảng cho nhiều kiến trúc hiện đại ngày nay.

Đặc điểm chính của kiến trúc RISC bao gồm:

- **Tập lệnh ít và đồng đều:** Mỗi lệnh có độ dài cố định (thường 32-bit), giúp giải mã lệnh nhanh hơn và đơn giản hóa phần cứng giải mã.
- **Chế độ địa chỉ đơn giản:** Chỉ hỗ trợ Load/Store để truy cập bộ nhớ; các phép toán được thực thi trực tiếp trên thanh ghi.
- **Nhiều thanh ghi đa năng:** Giảm thiểu số lần truy cập bộ nhớ, tăng tốc độ xử lý.
- **Pipeline hiệu quả:** Cấu trúc lệnh đều đặn, cố định tạo điều kiện thuận lợi cho xử lý đường ống (pipelining).

Ưu điểm:

- Tốc độ thực thi lệnh cao do mỗi lệnh hoàn thành trong một chu kỳ đồng hồ.
- Thiết kế mạch đơn giản hơn, dễ kiểm tra và tối ưu hóa.
- Tiêu thụ năng lượng thấp hơn so với CISC, phù hợp với thiết bị di động và nhúng.

Nhược điểm:

- Chương trình máy thường dài hơn do mỗi tác vụ phức tạp cần được phân rã thành nhiều lệnh đơn giản.
- Yêu cầu nhiều thanh ghi hơn, làm tăng diện tích chip.

Tiêu chí	RISC	CISC
Tập lệnh	Ít, đơn giản, cố định	Nhiều, phức tạp, biến thiên
Độ dài lệnh	Cố định	Thay đổi
Chu kỳ thực thi	1 chu kỳ/lệnh	Nhiều chu kỳ/lệnh
Bộ giải mã	Đơn giản	Phức tạp
Thanh ghi	Nhiều	Ít hơn
Pipeline	Dễ triển khai	Khó triển khai

Bảng 2: So sánh kiến trúc RISC và CISC

2.2 Tổ chức CPU đơn giản

Một CPU đơn giản bao gồm các khối chức năng chính sau:

- **ALU (Arithmetic Logic Unit):** Thực thi các phép toán số học (cộng, trừ) và logic (AND, XOR, NOT) trên dữ liệu đầu vào.
- **Register File:** Tập các thanh ghi tốc độ cao dùng lưu trữ tạm thời toán hạng và kết quả trung gian trong quá trình xử lý.
- **Control Unit:** Khối điều khiển, giải mã opcode và phát sinh các tín hiệu điều khiển tương ứng cho toàn bộ hệ thống.
- **Program Counter (PC):** Thanh ghi lưu địa chỉ của lệnh tiếp theo cần thực thi; tự động tăng sau mỗi chu kỳ lệnh.
- **Instruction Register (IR):** Lưu mã lệnh vừa được nạp từ bộ nhớ, phục vụ cho quá trình giải mã.
- **Memory:** Bộ nhớ lưu trữ cả chương trình (instruction memory) và dữ liệu (data memory).

Chu trình hoạt động Fetch – Decode – Execute (FDE):

1. **[Fetch]:** Nạp lệnh từ bộ nhớ vào IR theo địa chỉ PC.
2. **[Decode]:** Control Unit giải mã lệnh, phát tín hiệu điều khiển.
3. **[Execute]:** ALU thực thi phép toán, kết quả lưu vào Accumulator hoặc bộ nhớ.
4. **[Update]:** $PC \leftarrow PC + 1$ (hoặc địa chỉ nhảy nếu gặp JMP/SKZ).

Trong thiết kế RISC CPU đơn giản này, **Accumulator** đóng vai trò là thanh ghi trung gian chính, lưu giữ toán hạng và kết quả của các phép toán. **Address Mux** chịu trách nhiệm chọn lựa địa chỉ giữa Instruction Fetch và Operand Fetch tùy theo trạng thái hiện tại của bộ điều khiển. Toàn bộ chu kỳ hoạt động của CPU được đồng bộ hóa bởi xung clock, đảm bảo các khối phối hợp làm việc chính xác theo trình tự quy định.

2.3 Tập lệnh và mã hóa lệnh

CPU sử dụng định dạng lệnh 8-bit, trong đó 3-bit cao là opcode và 5-bit thấp là toán hạng (địa chỉ):

7	6	5	4	3	2	1	0
Opcode (3 bits)			Operand (5 bits)				

Bảng 3: Định dạng mã hóa lệnh 8-bit

Opcode	Mã	Hoạt động	Kết quả (Output)
HLT	000	Dừng hoạt động chương trình	inA
SKZ	001	Nếu $ALU = 0$, bỏ qua lệnh tiếp theo	inA
ADD	010	Cộng Accumulator với bộ nhớ	$inA + inB$
AND	011	AND Accumulator với bộ nhớ	$inA \text{ AND } inB$
XOR	100	XOR Accumulator với bộ nhớ	$inA \text{ XOR } inB$
LDA	101	Đọc giá trị từ địa chỉ vào Accumulator	inB
STO	110	Ghi Accumulator vào địa chỉ bộ nhớ	inA
JMP	111	Nhảy không điều kiện đến địa chỉ đích	inA

Bảng 4: Tập lệnh của RISC CPU

2.4 Ứng dụng thực tế của kiến trúc RISC

Kiến trúc RISC không chỉ là lý thuyết mà đã trở thành tiêu chuẩn công nghiệp, thống trị từ thị trường thiết bị di động đến siêu máy tính nhờ hiệu quả năng lượng và khả năng tối ưu hóa pipeline.

- **Hệ sinh thái ARM (Advanced RISC Machine):** Đây là kiến trúc RISC phổ biến nhất thế giới.
 - *Thiết bị di động:* Chiếm hơn 95% thị phần smartphone toàn cầu (Qualcomm Snapdragon, Apple A-series, MediaTek Dimensity).
 - *Máy tính cá nhân:* Bước ngoặt với Apple Silicon (M1, M2, M3 series), chứng minh RISC có thể vượt mặt CISC (x86) về hiệu năng trên mỗi Watt.
 - *Điện toán đám mây:* Các bộ vi xử lý như AWS Graviton hay Ampere Altra đang dần thay thế CPU truyền thống trong các trung tâm dữ liệu.
- **Kiến trúc MIPS (Microprocessor without Interlocked Pipeline Stages):** Dù không còn thống trị ở mảng người dùng cuối, MIPS vẫn giữ vai trò quan trọng:
 - *Thiết bị mạng:* Phổ biến trong các bộ định tuyến (Router), thiết bị chuyển mạch (Switch) của Cisco, Linksys và TP-Link.
 - *Hệ thống nhúng:* Sử dụng trong các bộ điều khiển công nghiệp, máy chơi game cổ điển (PS1, N64) và các thiết bị IoT yêu cầu độ ổn định cao.
 - *Giáo dục:* Là mô hình chuẩn để giảng dạy môn Kiến trúc máy tính nhờ cấu trúc pipeline 5 giai đoạn kinh điển.
- **RISC-V (Kiến trúc tập lệnh mở):** Được xem là tương lai của ngành bán dẫn nhờ tính tự do và không tốn phí bản quyền.
 - *Tùy biến chuyên dụng:* Các công ty như Western Digital và NVIDIA sử dụng RISC-V để thiết kế các bộ điều khiển (controller) tùy chỉnh cho ổ cứng và card đồ họa.

- *Trí tuệ nhân tạo (AI)*: RISC-V cho phép thêm các tập lệnh tăng tốc AI riêng biệt, giúp tối ưu hóa việc xử lý dữ liệu lớn.
- *Chủ quyền công nghệ*: Các quốc gia và tập đoàn lớn đang đầu tư vào RISC-V để giảm sự phụ thuộc vào các chuẩn đóng như ARM hay x86.

Kiến trúc	Thị trường trọng tâm	Đại diện tiêu biểu
ARM	Di động, Laptop, Server	iPhone, MacBook M3, AWS Graviton
MIPS	Mạng, Nhúng, Giáo dục	Router WiFi, Microchip PIC32
RISC-V	IoT, AI, Chip tùy biến	SiFive Intelligence, WD SweRV

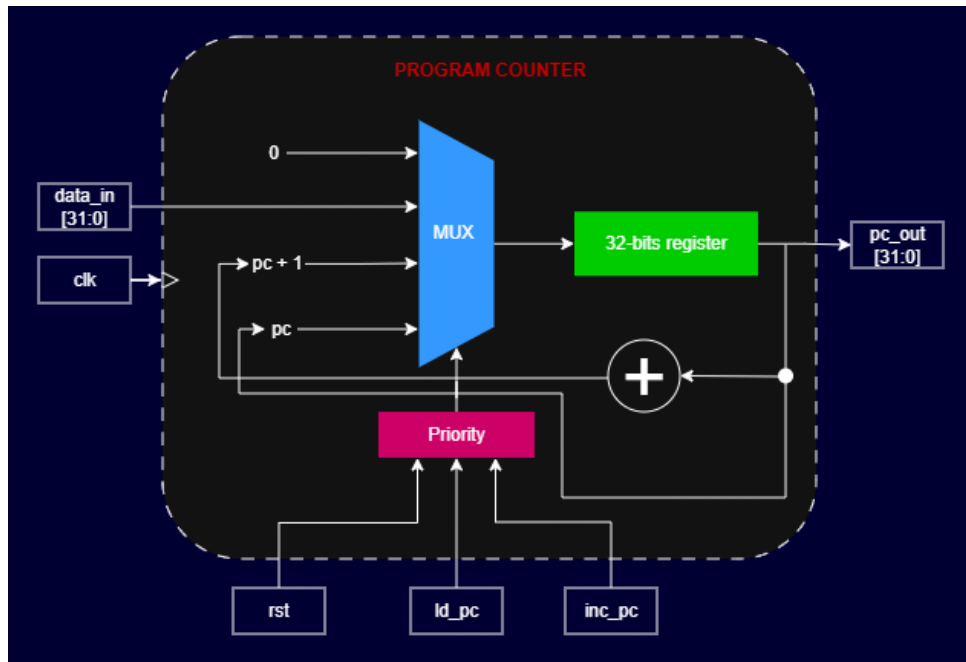
Bảng 5: Tóm tắt các hệ sinh thái RISC tiêu biểu

3 THIẾT KẾ

3.1 Các khối chức năng

3.1.1 Program Counter (PC)

Bộ đếm chương trình (Program Counter - PC) là một thanh ghi 32-bit có nhiệm vụ lưu trữ và cập nhật địa chỉ của câu lệnh tiếp theo sẽ được thực thi.



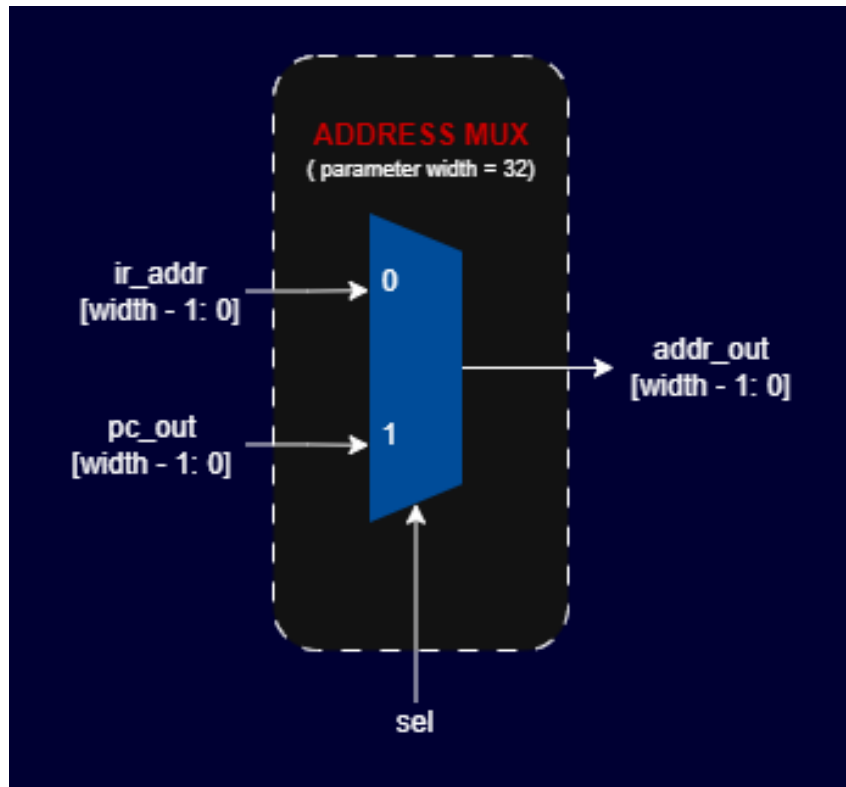
Hình 1: Sơ đồ khối bộ đếm chương trình (PC) 32-bit

Nguyên lý hoạt động: Khối PC hoạt động đồng bộ theo xung lên của clk và output được chọn bởi một MUX đi kèm với một mạch ưu tiên logic. Khối sẽ thay đổi trạng thái dựa trên các tín hiệu điều khiển với mức độ ưu tiên từ cao xuống thấp như sau:

- **Reset (*rst*):** Có mức ưu tiên cao nhất. Khi được kích hoạt, bộ đếm ngay lập tức bị xóa về giá trị 0 ($PC = 0$), đưa chương trình về trạng thái khởi tạo.
- **Nạp địa chỉ mới (*ld_pc*):** Được sử dụng khi thực thi lệnh rẽ nhánh (như lệnh JMP). PC sẽ nạp trực tiếp một giá trị địa chỉ 32-bit từ ngõ vào ($PC = data_in$).
- **Tăng bộ đếm (*inc_pc*):** Ở chế độ hoạt động tuần tự bình thường, bộ đếm sử dụng bộ cộng (Adder) để tự động tăng giá trị hiện tại lên 1 đơn vị ($PC = PC + 1$), giúp CPU trở tới lệnh tiếp theo.
- **Giữ nguyên trạng thái:** Nếu không có tín hiệu nào trong ba tín hiệu trên được kích hoạt, PC sẽ duy trì giá trị địa chỉ hiện tại.

3.1.2 Address Mux

Khối đa hợp địa chỉ (Address Mux) là một mạch tổ hợp đóng vai trò bộ chọn nguồn địa chỉ cung cấp cho bộ nhớ (Memory). Tùy thuộc vào giai đoạn hoạt động của CPU, khối này sẽ quyết định đường dẫn địa chỉ nào được phép đi vào bộ nhớ để truy xuất dữ liệu. Để đảm bảo tính linh hoạt, khối được thiết kế với tham số **width** (mặc định 32-bit), cho phép dễ dàng mở rộng không gian địa chỉ khi cần thiết.



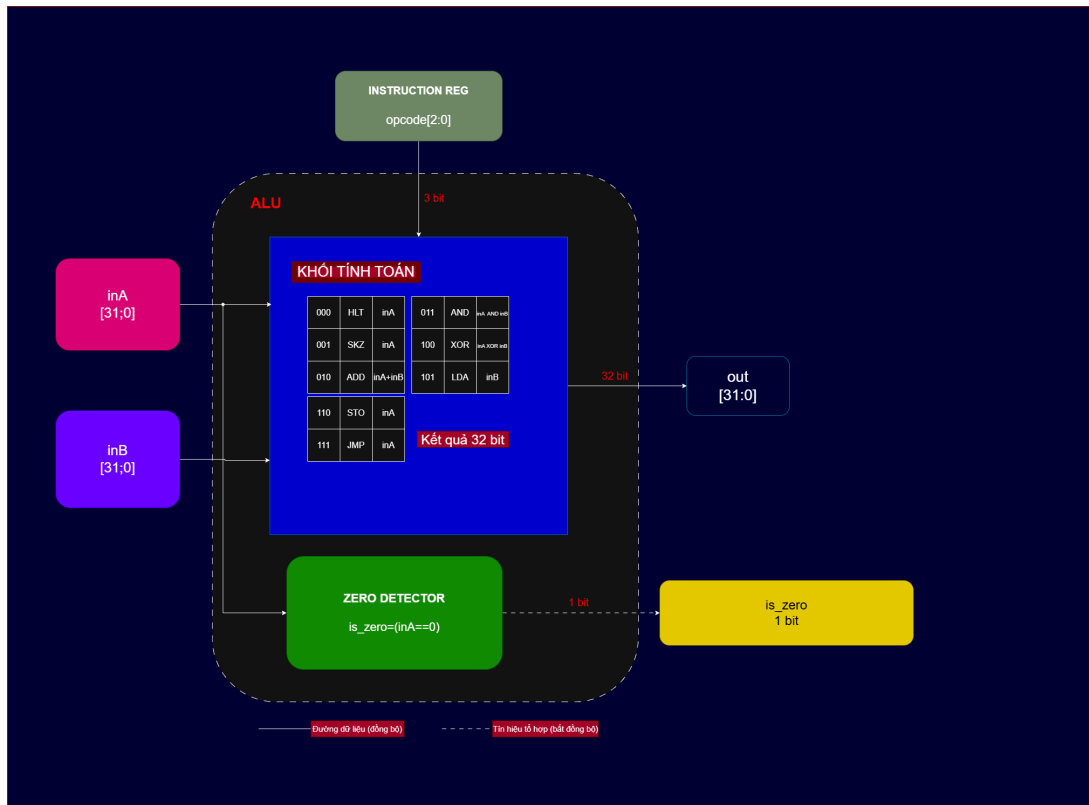
Hình 2: Sơ đồ khối bộ đa hợp địa chỉ (Address Mux)

Nguyên lý hoạt động: Hoạt động của Address Mux được điều khiển trực tiếp bởi tín hiệu *sel* từ khối Controller. Logic lựa chọn được thực hiện dựa trên trạng thái của tín hiệu điều khiển này:

- **Giai đoạn nạp lệnh ($sel = 1$):** Bộ đa hợp sẽ ưu tiên lựa chọn địa chỉ từ bộ đếm chương trình (*pc_out*). Lúc này, địa chỉ của lệnh tiếp theo sẽ được đưa đến bộ nhớ để thực hiện quá trình lấy lệnh (Instruction Fetch).
- **Giai đoạn thực thi ($sel = 0$):** Bộ đa hợp chuyển sang lựa chọn địa chỉ toán hạng từ thanh ghi lệnh (*ir_addr*). Điều này cho phép CPU truy xuất vào đúng vị trí của dữ liệu trong bộ nhớ để phục vụ cho các phép toán của ALU hoặc các lệnh lưu trữ dữ liệu.
- **Tính tham số hóa (Parameterization):** Việc sử dụng tham số **width** cho phép module tự động điều chỉnh độ rộng của tất cả các đường bus dữ liệu vào và ra (*ir_addr*, *pc_out*, *addr_out*). Điều này giúp thiết kế có tính linh hoạt cao và dễ dàng tái sử dụng trong các hệ thống có quy mô khác nhau.

3.1.3 ALU (Arithmetic Logic Unit)

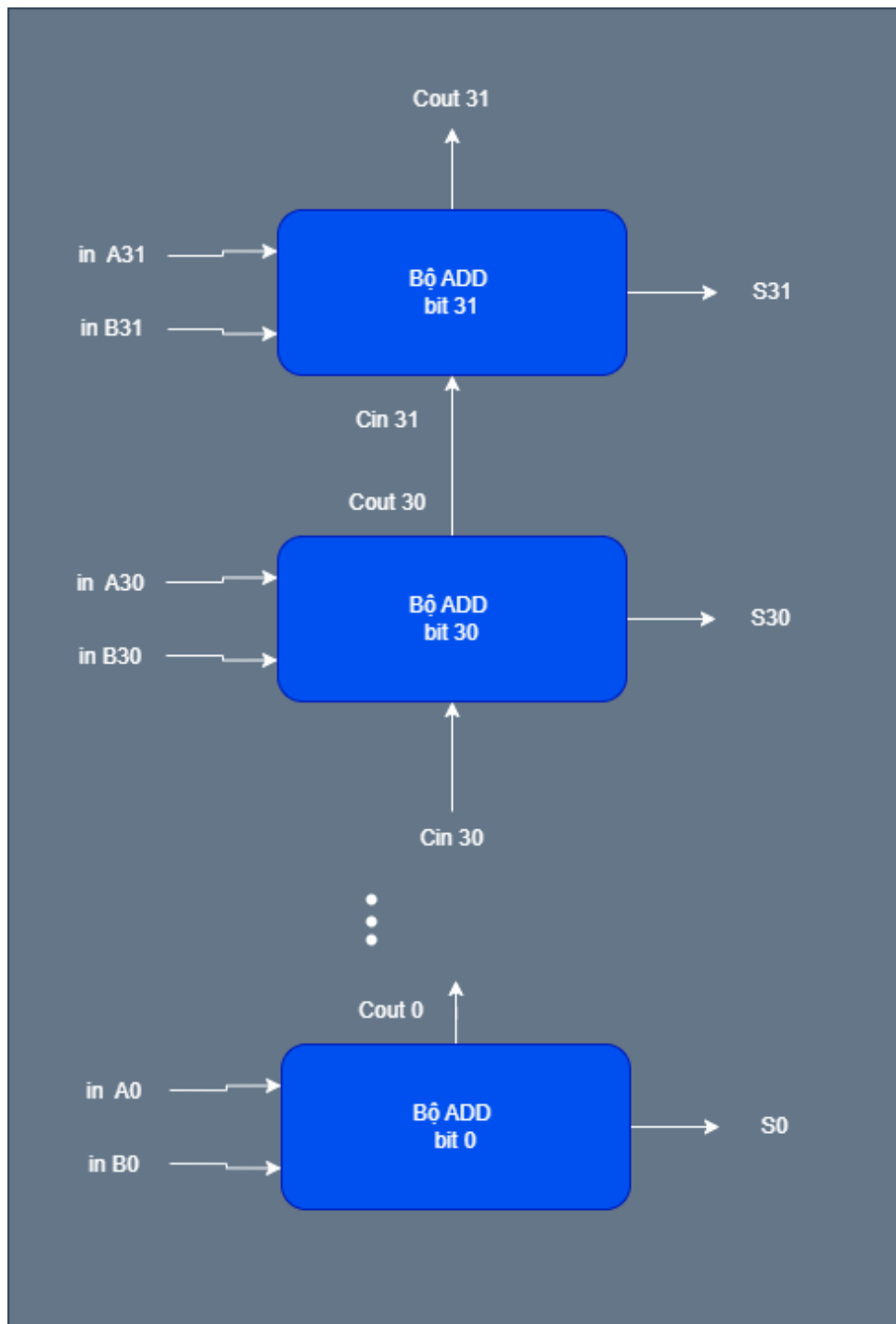
Đơn vị số học và logic (ALU) là thành phần thực thi các phép toán cốt lõi của CPU. Mặc dù hệ thống sử dụng định dạng mã hóa lệnh 8-bit (3-bit Opcode và 5-bit Operand), nhưng khối ALU được thiết kế để xử lý dữ liệu với độ rộng thực tế là **32-bit**. Điều này cho phép máy tính thực hiện các phép tính trên dải giá trị lớn, đáp ứng tiêu chuẩn của các kiến trúc máy tính hiện đại.



Hình 3: Sơ đồ khối tổng quát đơn vị ALU xử lý dữ liệu 32-bit

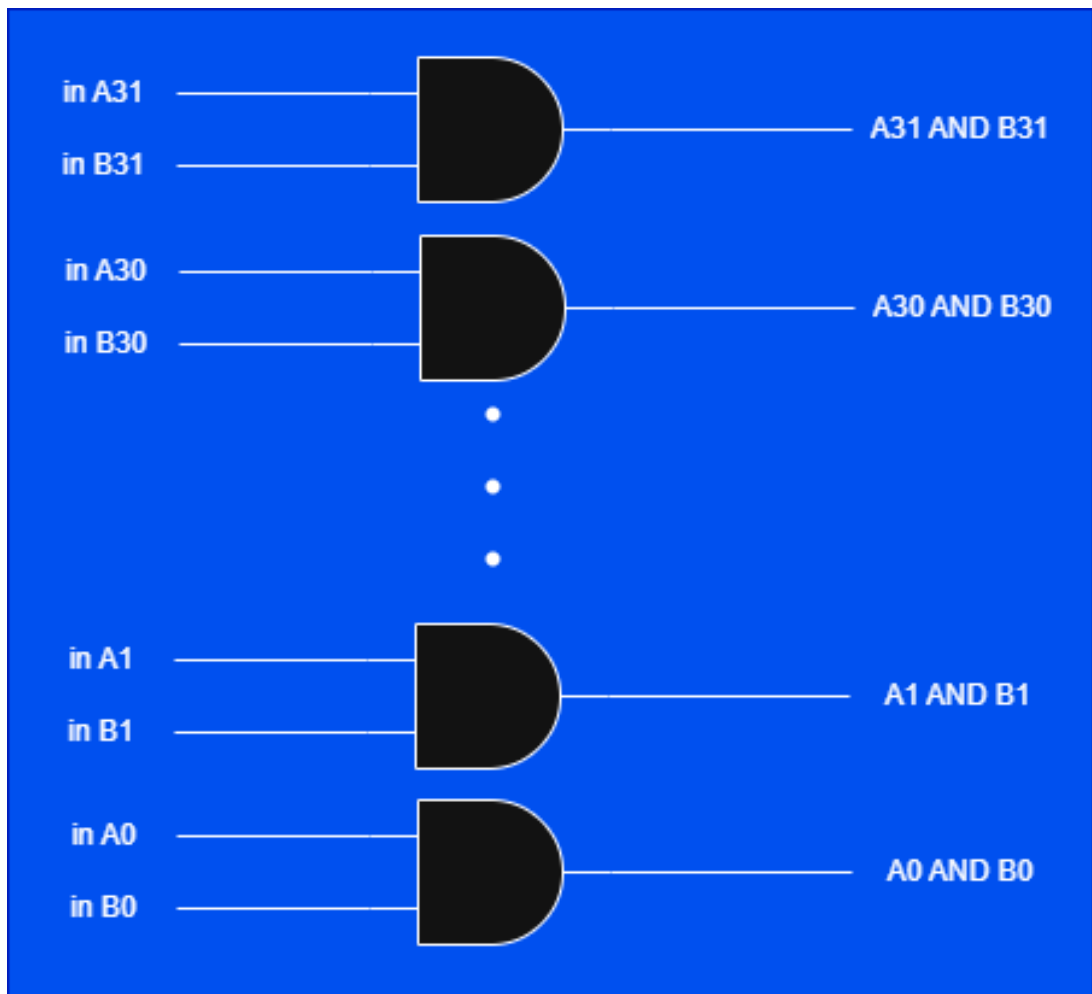
Nguyên lý thực thi các phép toán chi tiết: Dựa trên Opcode 3-bit từ khối điều khiển, ALU sẽ kích hoạt các mạch logic tương ứng để xử lý hai toán hạng 32-bit (inA và inB):

- **Phép cộng (ADD - Opcode 010):** Thực hiện cộng đại số hai giá trị 32-bit. Kết quả $ALU_Out = inA + inB$. Phép toán này sử dụng bộ cộng đầy đủ (Full Adder) 32-bit để xử lý dữ liệu từ Accumulator và bộ nhớ.



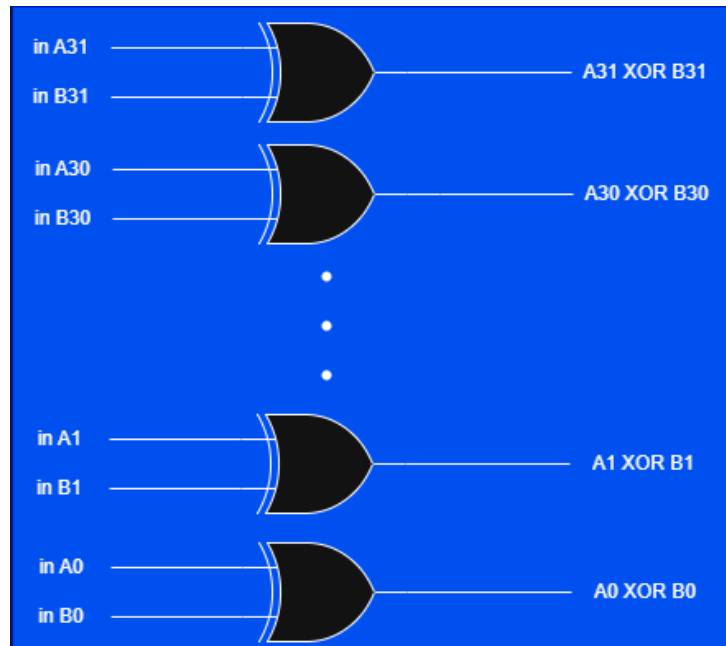
Hình 4: Sơ đồ khối logic cho phép toán ADD 32-bit

- **Phép logic AND (Opcode 011):** Thực hiện phép toán bitwise AND giữa inA và inB trên toàn bộ 32 bit. Kết quả đầu ra chỉ bằng 1 tại các vị trí bit mà cả hai đầu vào đều bằng 1.



Hình 5: Sơ đồ khối logic cho phép toán AND 32-bit

- **Phép logic XOR (Opcode 100):** Thực hiện phép toán bitwise XOR giữa hai toán hạng 32-bit. Phép toán này đặc biệt hữu ích trong việc kiểm tra sự khác biệt hoặc đảo bit dữ liệu.



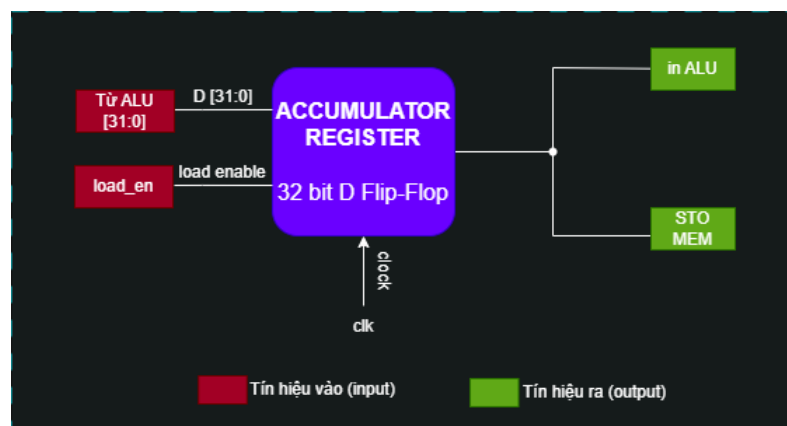
Hình 6: Sơ đồ khối logic cho phép toán XOR 32-bit

Tín hiệu Zero Flag (*is_zero*): Một đầu ra quan trọng khác của ALU là tín hiệu *is_zero* được tính toán bất đồng bộ. Nếu toàn bộ 32 bit của kết quả đầu ra đều bằng 0, tín hiệu này sẽ được kéo lên mức logic 1. Đây là cơ sở để thực hiện lệnh nhảy có điều kiện **SKZ** (Skip if Zero).

3.1.4 Register (Accumulator / Instruction Register)

a. Accumulator Register

Accumulator Register là một thanh ghi 32-bit đóng vai trò trung tâm trong luồng dữ liệu của kiến trúc RISC này. Nó chịu trách nhiệm lưu trữ các kết quả trung gian từ ALU và cung cấp toán hạng cho các phép toán tiếp theo hoặc ghi ra bộ nhớ.



Hình 7: Sơ đồ khối Accumulator Register

Sơ đồ khối của Accumulator Register gồm các thành phần giao tiếp chính:

- Tín hiệu vào (Input – màu tím):

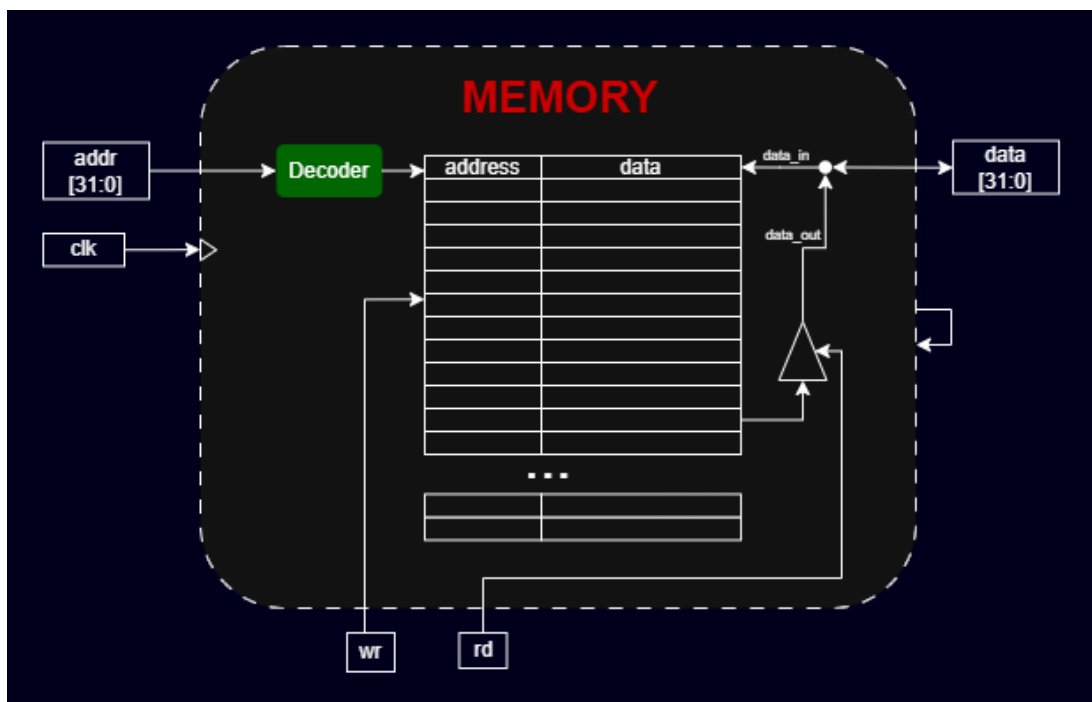
- D [31:0]: Dữ liệu 32-bit nhận từ ngõ ra của khối ALU.
- load_en: Tín hiệu cho phép ghi vào Accumulator. Dữ liệu chỉ được cập nhật khi tín hiệu này ở mức tích cực.
- clk: Xung clock đồng bộ hóa quá trình lưu trữ dữ liệu theo sườn xung.

- **Tín hiệu ra (Output – màu xanh lá):**

- inA: Cấp ngược lại cho ALU làm toán hạng đầu vào thứ nhất cho các phép tính số học và logic.
- ST0: Dữ liệu 32-bit được trích xuất để ghi vào bộ nhớ dữ liệu khi thực thi lệnh STO.

3.1.5 Memory

Bộ nhớ (Memory) đóng vai trò là kho lưu trữ trung tâm cho cả lệnh chương trình và dữ liệu. Điểm đặc biệt trong thiết kế này là việc sử dụng một cổng dữ liệu lưỡng cực duy nhất (Single bidirectional data port) để tối ưu hóa tài nguyên, đồng thời yêu cầu một cơ chế điều phối chặt chẽ để đảm bảo không xảy ra hiện tượng đọc và ghi cùng một lúc.



Hình 8: Sơ đồ cấu trúc nội bộ khối Memory 32-bit

Nguyên lý hoạt động: Khối Memory hoạt động đồng bộ với xung lên của clk và được quản lý bởi các tín hiệu điều khiển từ Controller. Kiến trúc nội bộ của khối bao gồm các thành phần xử lý logic sau:

- **Giải mã địa chỉ (Address Decoding):** Địa chỉ 32-bit từ ngõ vào sẽ đi qua khối giải mã (Decoder). Khối này có nhiệm vụ xác định chính xác vị trí hàng và cột trong mảng ô nhớ (Memory Array) để chuẩn bị cho các thao tác truy xuất dữ liệu.

- **Ghi dữ liệu (Write Logic - $wr = 1$):** Quá trình ghi diễn ra đồng bộ tại xung lên của clk . Khi tín hiệu wr được kích hoạt, dữ liệu từ cổng $data$ bên ngoài sẽ đi theo nhánh $data_in$ để chốt vào ô nhớ đã được chọn bởi bộ giải mã.
- **Đọc dữ liệu và Trở kháng cao (Read Logic & High-Z):** Dữ liệu từ ô nhớ được đưa ra đường $data_out$ dẫn tới bộ đệm 3 trạng thái (Tri-state Buffer).
 - Khi $rd = 1$: Bộ đệm mở cho phép dữ liệu từ $data_out$ tràn ra cổng giao tiếp bên ngoài.
 - Khi không có lệnh đọc: Bộ đệm sẽ đưa ngõ ra về trạng thái trở kháng cao (Z), giúp "ngắt kết nối" bộ nhớ khỏi bus dữ liệu để nhường đường cho các khối khác ghi vào.
- **Ràng buộc an toàn:** Thiết kế đảm bảo tính toàn vẹn bằng cách ngăn chặn tuyệt đối việc đọc và ghi dữ liệu đồng thời, tránh gây xung đột và hư hỏng dữ liệu trên bus dùng chung.

4 HIỆN THỰC

4.1 Hiện thực ALU

Khối ALU được hiện thực bằng ngôn ngữ mô tả phần cứng Verilog. Module nhận hai toán hạng đầu vào 32-bit (*inA*, *inB*) và tín hiệu điều khiển 3-bit (*opcode*). Dựa vào *opcode*, khối ALU sẽ sử dụng cấu trúc **case** để định tuyến dữ liệu đầu ra hoặc thực hiện các phép toán số học/logic tương ứng.

Mã nguồn Verilog hiện thực khối ALU:

Listing 1: Mã nguồn hiện thực module ALU

```
1 module ALU (  
2     input  [31:0] inA,  
3     input  [31:0] inB,  
4     input  [2:0]  opcode,  
5     output reg [31:0] out,  
6     output          is_zero  
7 );  
8  
9     localparam HLT = 3'b000, SKZ = 3'b001, ADD = 3'b010, AND = 3'  
10         b011,  
11         XOR = 3'b100, LDA = 3'b101, STO = 3'b110, JMP = 3'  
12         b111;  
13  
14     always @(*) begin  
15         case (opcode)  
16             HLT, SKZ, JMP, STO : out = inA;  
17             ADD : out = inA + inB;  
18             AND : out = inA & inB;  
19             XOR : out = inA ^ inB;  
20             LDA : out = inB;  
21             default: out = inA;  
22         endcase  
23     end  
24  
25     assign is_zero = (inA == 32'b0);  
26 endmodule
```

Mô tả logic xử lý chi tiết:

- **Nhóm lệnh tính toán (ADD, AND, XOR):** ALU thực hiện trực tiếp các phép toán tương ứng giữa hai toán hạng *inA* và *inB*. Việc tổng hợp (synthesis) các toán tử +, &, ^ sẽ tự động tạo ra các bộ cộng và cổng logic phần cứng tương ứng.

- **Lệnh nạp dữ liệu (LDA):** Dữ liệu từ ngõ vào `inB` (thường nối với bộ nhớ dữ liệu) được truyền trực tiếp ra `out` để chuẩn bị nạp vào thanh ghi Accumulator.
- **Nhóm lệnh truyền thẳng (HLT, SKZ, JMP, STO):** Các lệnh này không yêu cầu tính toán. ALU sẽ đóng vai trò như một đường dẫn trong suốt, truyền trực tiếp giá trị của `inA` ra ngõ `out`. Việc gom nhóm các case này giúp phần mềm tổng hợp tối ưu hóa bộ dồn kênh (MUX), làm giảm diện tích mạch và rút ngắn trễ truyền (propagation delay).
- **Tín hiệu cờ `is_zero`:** Tín hiệu này được thiết kế bằng lệnh `assign` nằm ngoài khối `always`, giúp nó hoạt động hoàn toàn độc lập và bất đồng bộ. Tín hiệu này giám sát liên tục ngõ vào `inA`; nếu `inA = 0`, cờ sẽ lập tức bật lên mức 1 mà không cần chờ phép toán bên trong ALU hoàn tất, đáp ứng đúng đặc tả của lệnh nhảy SKZ.

4.2 Hiện thực Register

4.2.1 Hiện thực Accumulator Register

Trong kiến trúc CPU RISC này, Thanh ghi tích lũy (Accumulator Register) đóng vai trò trung tâm trong luồng dữ liệu, chịu trách nhiệm lưu trữ toán hạng và kết quả tính toán 32-bit từ khối ALU. Thanh ghi được thiết kế dưới dạng mạch dây (Sequential Logic) với tín hiệu Reset đồng bộ nhằm đảm bảo tính ổn định và nhịp nhàng của toàn hệ thống.

Mã nguồn Verilog hiện thực module Accumulator Register:

Listing 2: Mã nguồn hiện thực module Accumulator Register

```

1 module Accumulator (
2     input      clk,
3     input      rst,
4     input      load_en,
5     input  [31:0] D,
6     output reg [31:0] Q      //(inA cho ALU)
7 );
8
9
10
11 always @(posedge clk) begin
12     if (rst) begin
13         Q <= 32'b0;
14     end
15     else if (load_en) begin
16         Q <= D;
17     end
18 end
19

```

Mô tả logic xử lý:

- **Đồng bộ hóa tuyệt đối:** Khối `always` chỉ nhảy với sườn lên của xung nhịp (`posedge clk`). Tín hiệu `rst` được kiểm tra ngay bên trong cấu trúc điều kiện của khối `always`, đảm bảo mạch chỉ thực hiện việc xóa dữ liệu (`Q <= 0`) khi có sườn xung clock tác động (Reset đồng bộ).
- **Cơ chế chốt dữ liệu:** Khi tín hiệu cho phép `load_en` ở mức logic 0, thanh ghi tự động duy trì trạng thái cũ. Điều này giúp hệ thống giữ vững kết quả tính toán, ngăn chặn việc dữ liệu bị ghi đè bởi các tín hiệu rác trên đường dẫn từ ALU trong các chu kỳ máy không có yêu cầu cập nhật.

5 KIỂM THỬ VÀ ĐÁNH GIÁ

5.1 Testbench từng module

5.1.1 Testbench và Kết quả mô phỏng ALU

a. Mô tả kịch bản kiểm chứng (Testbench)

Môi trường mô phỏng (Testbench) được xây dựng nhằm kiểm tra toàn diện chức năng của khối số học và logic (ALU). Kịch bản kiểm tra (stimulus) được thiết kế với các mục tiêu cụ thể:

- **Quét toàn bộ tập lệnh:** Lần lượt cấp các tín hiệu opcode từ 000 đến 111 kết hợp với các giá trị inA và inB khác nhau để kiểm tra ngõ ra out.
- **Kiểm tra tính bất đồng bộ của is_zero:** Thử nghiệm thay đổi inA đột ngột để quan sát sự phản hồi ngay lập tức của cờ is_zero mà không cần phụ thuộc vào xung clock hay sự thay đổi của opcode.
- **Bẫy lỗi (Corner Case):** Tạo test case số 9 thực hiện phép cộng số bù 2: $10 + (-10)$. Kết quả phép tính bằng 0 nhưng toán hạng inA $\neq 0$. Mục tiêu là chứng minh cờ is_zero chỉ phụ thuộc vào inA theo đúng đặc tả của lệnh nhảy SKZ.

b. Mã nguồn Testbench (Verilog)

Listing 3: Mã nguồn Testbench kiểm tra khối ALU

```
1  `timescale 1ns / 1ps
2
3  module tb_ALU();
4
5
6      reg  [31:0] inA;
7      reg  [31:0] inB;
8      reg  [2:0]  opcode;
9
10     wire [31:0] out;
11     wire          is_zero;
12
13
14     localparam HLT = 3'b000, SKZ = 3'b001, ADD = 3'b010, AND = 3'
15         b011,
16                 XOR = 3'b100, LDA = 3'b101, STO = 3'b110, JMP = 3'
17         b111;
18
19     ALU uut (
20         .inA(inA),
```

```

20         .inB(inB),
21         .opcode(opcode),
22         .out(out),
23         .is_zero(is_zero)
24     );
25
26
27     initial begin
28
29         $display("-----");
30         $display("Time|Opcode|inA(hex)|inB(hex)|out(hex)|is_zero");
31         ;
32         $display("-----");
33
34         $monitor("%8t|%b|%h|%h|%h|%b",
35                 $time, opcode, inA, inB, out, is_zero);
36
37
38         inA = 32'd0;
39         inB = 32'd0;
40         opcode = HLT;
41         #10;
42
43         // TEST CASE 1:  ADD
44
45         inA = 32'd15;
46         inB = 32'd25;
47         opcode = ADD;
48         #10;
49
50
51         // TEST CASE 2:  AND
52
53         inA = 32'hFFFF0000;
54         inB = 32'hFF00FF00;
55         opcode = AND;
56         #10;
57
58
59         // TEST CASE 3:  XOR
60
61         inA = 32'hAAAAAAAA;
62         inB = 32'h55555555;
63         opcode = XOR;

```

```

64      #10;
65
66
67      // TEST CASE 4:  LDA (Load inB)
68
69      inA = 32'd99;
70      inB = 32'h1234ABCD;
71      opcode = LDA;
72      #10;
73
74
75      // TEST CASE 5:  inA (HLT, SKZ, STO, JMP)
76
77      inA = 32'hCAFEBABE;
78      inB = 32'd0;
79      opcode = STO;
80      #10;
81
82
83      // TEST CASE 6:  is_zero
84
85      inA = 32'd0;
86      inB = 32'd999;
87      opcode = ADD;
88      #10;
89
90
91      // TEST CASE 7:  SKZ with inA = 0
92
93      inA = 32'd0;
94      inB = 32'hFFFFFFFF;
95      opcode = SKZ;
96      #10;
97
98
99      // TEST CASE 8:  is_zero nay su thay doi inA ko thay doi
100      opcode
101
102      inA = 32'd5;
103      #5;
104      inA = 32'd0;
105      #5;
106
107      // TEST CASE 9:  inA != 0 but result=0

```

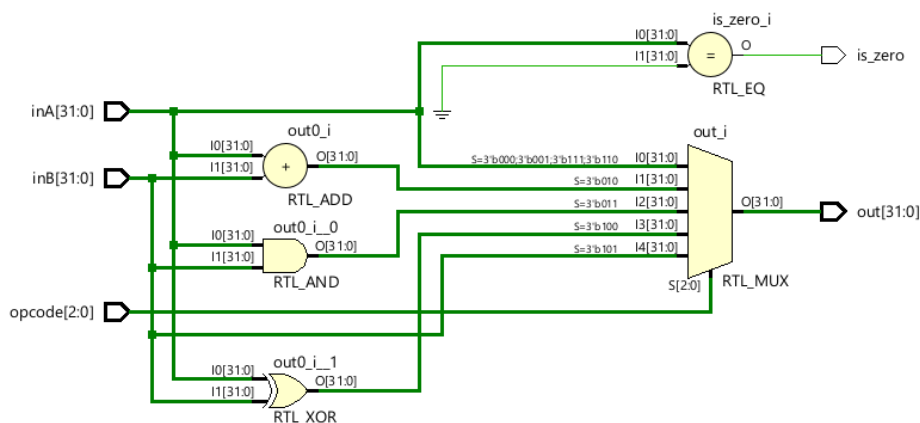
```

108 inA = 32'd10;
109 inB = -32'd10;
110 opcode = ADD;
111 #10;
112
113
114     $display("-----");
115     $display("HOAN THANH MO PHONG!");
116     $finish;
117 end
118
119 endmodule

```

c. Sơ đồ mạch RTL (RTL Schematic)

Quá trình tổng hợp (Synthesis) từ mã nguồn Verilog đã tạo ra sơ đồ mạch logic của ALU. Sơ đồ cho thấy bộ dồn kênh (Multiplexer) được tối ưu hóa để định tuyến dữ liệu dựa trên tín hiệu opcode, và một bộ dò cờ Zero (Zero Flag Detector) được kết nối trực tiếp vào bus inA.



Hình 9: Sơ đồ mạch logic (RTL Schematic) của khối ALU

d. Kết quả xuất ra màn hình (Console Output)

Thông qua các hàm hệ thống `$monitor` và `$display`, toàn bộ sự kiện thay đổi tín hiệu được ghi nhận lại theo thời gian thực.

Thời gian	Opcode	inA (hex)	inB (hex)	out (hex)	is_zero
0	000	00000000	00000000	00000000	1
10000	010	0000000f	00000019	00000028	0
20000	011	ffff0000	ff00ff00	ff000000	0
30000	100	aaaaaaaa	55555555	ffffffff	0
40000	101	00000063	1234abcd	1234abcd	0
50000	110	cafeba5e	00000000	cafeba5e	0
60000	010	00000000	000003e7	000003e7	1
70000	001	00000000	ffffffff	00000000	1
80000	001	00000005	ffffffff	00000005	0
85000	001	00000000	ffffffff	00000000	1
90000	010	0000000a	ffffffff	00000000	0

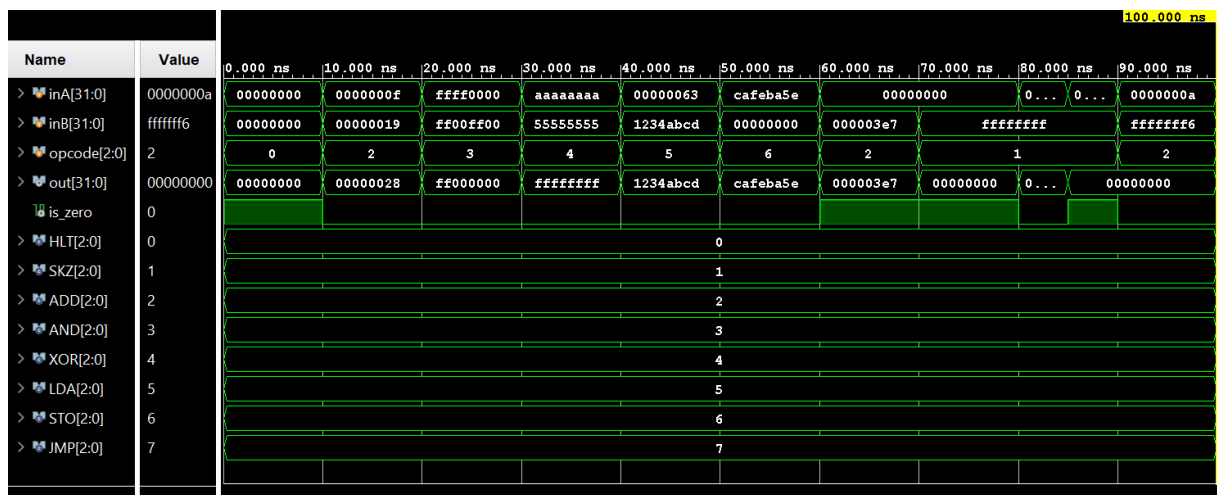
Hoan thành mô phỏng!

Hình 10: Kết quả chạy Testbench khối ALU hiển thị trên Console

Dựa vào bảng kết quả trên Console, các phép toán ADD, AND, XOR và các lệnh nạp dữ liệu hoạt động hoàn toàn chính xác. Tại thời điểm 90000 ps, phép toán $10 + (-10)$ cho ra $out = 00000000$ nhưng $is_zero = 0$ (vì $inA = 0000000a$), minh chứng thiết kế đã đúng tính chất bất đồng bộ.

e. Giải đồ sóng (Waveform)

Giải đồ thời gian cung cấp cái nhìn trực quan nhất về sự thay đổi của các tín hiệu điện.



Hình 11: Giải đồ thời gian (Waveform) minh họa hoạt động của ALU

Trên giải đồ sóng, có thể dễ dàng nhận thấy cờ is_zero chuyển trạng thái ngay lập tức khi đường bus inA chuyển về mức 0. Đường out cũng phản hồi chính xác theo sự phối hợp của inA , inB và sự thay đổi của $opcode$.

5.1.2 Testbench và Kết quả mô phỏng Accumulator Register

a. Mô tả kịch bản kiểm chứng (Testbench)

Môi trường mô phỏng được thiết lập nhằm đánh giá khả năng lưu trữ, chốt dữ liệu và đặc tính Reset đồng bộ của Thanh ghi tích lũy. Kịch bản bơm tín hiệu (stimulus) bao gồm các trường hợp:

- Kích hoạt tín hiệu **rst** độc lập để kiểm tra mạch có thực sự chờ đến sườn lên của clk mới thực hiện xóa dữ liệu hay không.

- Đưa dữ liệu ngẫu nhiên vào ngõ D nhưng tắt tín hiệu `load_en` để kiểm chứng tính năng giữ trạng thái cũ (chốt dữ liệu).
- Bật đồng thời `load_en` và `rst` để kiểm tra mức độ ưu tiên của tín hiệu điều khiển.

b. Mã nguồn Testbench (Verilog)

Listing 4: Mã nguồn Testbench kiểm tra Accumulator Register

```

1  `timescale 1ns / 1ps
2
3  module tb_Accumulator();
4
5      reg clk;
6      reg rst;
7      reg load_en;
8      reg [31:0] D;
9      wire [31:0] Q;
10
11
12      Accumulator uut (
13          .clk(clk),
14          .rst(rst),
15          .load_en(load_en),
16          .D(D),
17          .Q(Q)
18      );
19
20
21      always #5 clk = ~clk;
22
23      initial begin
24
25          $display("-----");
26          $display("Time|rst|load_en|D(hex)|Q(hex)");
27          $display("-----");
28          $monitor("%10t|%b|%b|h|h", $time, rst, load_en, D, Q);
29
30
31          clk = 0; rst = 0; load_en = 0; D = 32'h0;
32
33
34          #12; rst = 1;
35          #10.1; rst = 0;
36
37
38          #10.1; D = 32'hAAAA_5555; load_en = 1;

```

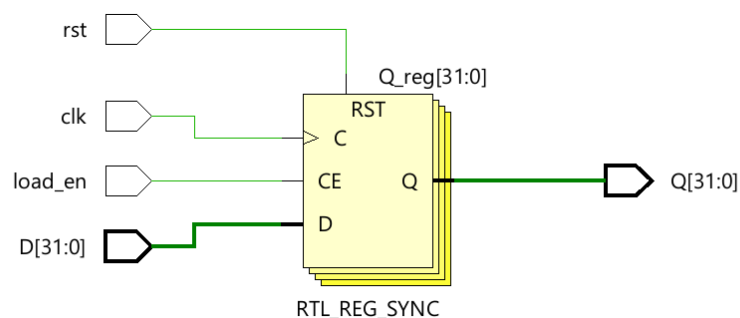
```

39      #10.1; load_en = 0;
40
41
42      #10.1; D = 32'h1234_ABCD;
43      #10.1;
44
45
46      #10.1; load_en = 1;
47      #10.1; load_en = 0;
48
49
50      #10.1; D = 32'hFFFF_FFFF; load_en = 1; rst = 1;
51      #10.1; rst = 0; load_en = 0;
52
53      #20;
54      $display("-----");
55      $display("Hoan thanh mo phong Accumulator!");
56      $finish;
57  end
58
59  endmodule

```

c. Sơ đồ mạch RTL (RTL Schematic)

Sơ đồ mạch logic sau quá trình tổng hợp cho thấy Accumulator được hiện thực bằng một tập hợp các Flip-Flop D (D-FF) có tích hợp bộ dồn kênh (MUX) điều khiển qua chân Enable và Reset.



Hình 12: Sơ đồ mạch logic (RTL Schematic) của khối Accumulator

d. Kết quả xuất ra màn hình (Console Output)

Quá trình thay đổi tín hiệu được tự động ghi nhận trên cửa sổ Console.

Thời gian	rst	load_en	D (hex)	Q (hex)
	0	0	0	00000000
12000	1	1	0	00000000
15000	1	1	0	00000000
22000	0	1	0	00000000
32000	0	0	1	aaaa5555
35000	0	0	1	aaaa5555
42000	0	0	0	aaaa5555
52000	0	0	0	1234abcd
73000	0	1	1	1234abcd
75000	0	1	1	1234abcd
83000	0	0	0	1234abcd
93000	1	1	1	ffffffff
95000	1	1	1	ffffffff
103000	0	0	0	00000000

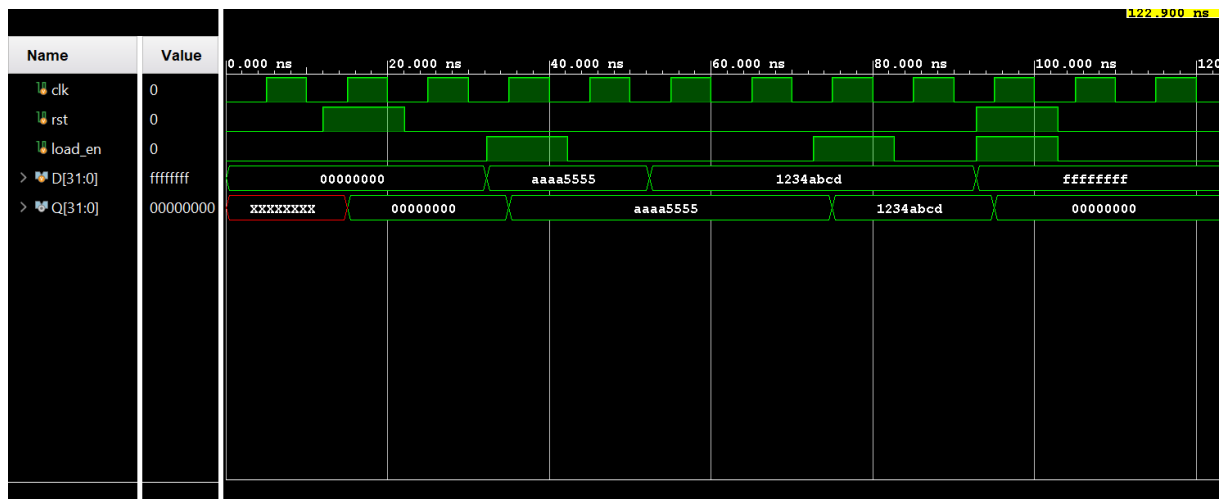
Hoan thành mô phỏng Accumulator!

Hình 13: Kết quả chạy Testbench Accumulator trên Console

Dựa vào log hiển thị, ngõ ra Q chỉ phản hồi với ngõ vào D khi có lệnh `load_en`. Trong trường hợp cả `rst` và `load_en` cùng ở mức cao, thanh ghi lập tức ưu tiên đưa dữ liệu về mức 0, tuân thủ đúng mức ưu tiên trong cấu trúc `if-else` đã thiết kế.

e. Giải đồ sóng (Waveform)

Giải đồ thời gian cung cấp minh chứng trực quan nhất cho tính đồng bộ của thanh ghi.



Hình 14: Giải đồ thời gian (Waveform) mô phỏng khối Accumulator

Quan sát trên giải đồ, tại thời điểm tín hiệu `rst` chuyển lên mức 1, dữ liệu ngõ ra Q không lập tức bị xóa đi mà duy trì trạng thái cho đến khi gặp cạnh lên tiếp theo của tín hiệu `clk`. Điều này chứng minh quá trình loại bỏ sự kiện `posedge rst` khỏi danh sách độ nhạy đã xử lý thành công yêu cầu thiết kế Reset đồng bộ.

6 KẾT LUẬN

6.1 Kết quả đạt được

Thông qua quá trình nghiên cứu và thực hiện đồ án, nhóm đã đạt được các kết quả trọng tâm sau:

- **Thiết kế hệ thống:** Hoàn thành việc thiết kế và mô phỏng hệ thống RISC CPU đơn giản dựa trên ngôn ngữ mô tả phần cứng Verilog. Hệ thống hỗ trợ tập lệnh gồm 8 lệnh nền tảng (HLT, SKZ, ADD, AND, XOR, LDA, STO, JMP), đáp ứng các yêu cầu về tính toán số học, logic và điều khiển luồng.
- **Kiến trúc phân khối:** Xây dựng thành công các khối chức năng chuyên biệt bao gồm: Program Counter, Address Mux, ALU, Controller (với FSM 8 trạng thái), Accumulator Register, Instruction Register và Memory.
- **Năng lực chuyên môn:** củng cố tư duy về sự tương tác dữ liệu và tín hiệu điều khiển trong kiến trúc máy tính thực tế. Đồng thời, nhóm đã làm chủ các kỹ năng lập trình HDL, xây dựng môi trường kiểm chứng (Testbench) và phân tích giản đồ thời gian (Waveform) để tối ưu hóa mạch số.

6.2 Khó khăn và Thách thức

Trong quá trình triển khai, nhóm đã đối mặt và giải quyết các thách thức kỹ thuật sau:

- **Mô hình hóa bộ điều khiển:** Việc thiết kế máy trạng thái hữu hạn (FSM) cho Controller đòi hỏi sự chính xác tuyệt đối trong việc phối hợp 7 tín hiệu đầu ra cho từng Opcode. Nhóm đã khắc phục bằng cách chuẩn hóa bảng trạng thái chi tiết trước khi hiện thực mã nguồn.
- **Kiểm chứng và Debug:** Quá trình xác minh lỗi trên quy mô hệ thống phức tạp đòi hỏi khả năng phân tích Waveform sâu. Giải pháp được áp dụng là chiến lược "Unit Testing" – kiểm tra độc lập từng module trước khi tiến hành tích hợp hệ thống (Integration).
- **Đồng bộ hóa tín hiệu:** Đảm bảo tính toàn vẹn của dữ liệu và tránh các hiện tượng xung đột trạng thái (race condition) thông qua việc kiểm soát chặt chẽ các thời điểm lấy mẫu dữ liệu (Timing).

6.3 Hướng phát triển và Mở rộng

Nhằm nâng cao hiệu suất và tính ứng dụng của kiến trúc, nhóm đề xuất các hướng phát triển tiếp theo:

- **Nâng cấp tập lệnh (ISA):** Mở rộng khả năng xử lý bằng cách tích hợp các phép toán phức tạp (nhân/chia), xử lý số dấu phẩy động và các chế độ địa chỉ gián tiếp.

- **Tối ưu hóa kiến trúc Pipeline:** Triển khai cấu trúc đường ống nhiều giai đoạn (IF, ID, EX, MEM, WB) nhằm tăng cường thông lượng (throughput) và khai thác tối đa tài nguyên phần cứng.
- **Triển khai phần cứng thực tế:** Thực hiện tổng hợp và nạp thiết kế lên bo mạch FPGA (như Arty Z7) để đánh giá các thông số về tài nguyên thực tế và tần số hoạt động tối đa. Đồng thời phát triển trình hợp dịch (Assembler) riêng để hỗ trợ lập trình mức cao cho CPU.

6.4 Bảng phân chia công việc

Bảng 6: Bảng phân chia công việc thực hiện đồ án

STT	Họ và tên	MSSV	Nhiệm vụ	Hoàn thành
1	PHẠM ĐỖ HỒNG PHÚC	2510359	Thiết kế & code Verilog khối ALU và Accumulator Register. Viết testbench tương ứng. Tổng hợp và ghi báo cáo latex.	100%
2	TRẦN TIẾN THÀNH	2413182	Thiết kế & code Verilog khối Controller. Tích hợp Top-level và viết testbench toàn hệ thống.	100%
3	VÕ LÊ PHÚC THỊNH	2510382	Thiết kế & code Verilog khối Memory, PC, Address Mux và IR. Viết testbench tương ứng.	100%